

19.1 INTRODUCTION

Intel® Advanced Matrix Extensions (Intel® AMX) is a new 64-bit programming paradigm consisting of two components: a set of 2-dimensional registers (tiles) representing sub-arrays from a larger 2-dimensional memory image, and an accelerator able to operate on tiles, the first implementation is called TMUL (tile matrix multiply unit).

An Intel AMX implementation enumerates to the programmer how the tiles can be programmed by providing a palette of options. Two palettes are supported; palette 0 represents the initialized state, and palette 1 consists of 8 KB of storage spread across 8 tile registers named TMM0..TMM7. Each tile has a maximum size of 16 rows x 64 bytes, (1 KB), however the programmer can configure each tile to smaller dimensions appropriate to their algorithm. The tile dimensions supplied by the programmer (rows and bytes_per_row, i.e., **colsb**) are metadata that drives the execution of tile and accelerator instructions. In this way, a single instruction can launch autonomous multi-cycle execution in the tile and accelerator hardware. The palette value (**palette_id**) and metadata are held internally in a tile related control register (TILECFG). The TILECFG contents will be commensurate with that reported in the palette_table (see “CPUID—CPU Identification” in the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 2A for a description of the available parameters).

Intel AMX is an extensible architecture. New accelerators can be added, or the TMUL accelerator may be enhanced to provide higher performance. In these cases, the state (TILEDATA) provided by tiles may need to be made larger, either in one of the metadata dimensions (more rows or colsb) and/or by supporting more tile registers (names). The extensibility is carried out by adding new palette entries describing the additional state. Since execution is driven through metadata, an existing Intel AMX binary could take advantage of larger storage sizes and higher performance TMUL units by selecting the most powerful palette indicated by CPUID and adjusting loop and pointer updates accordingly.

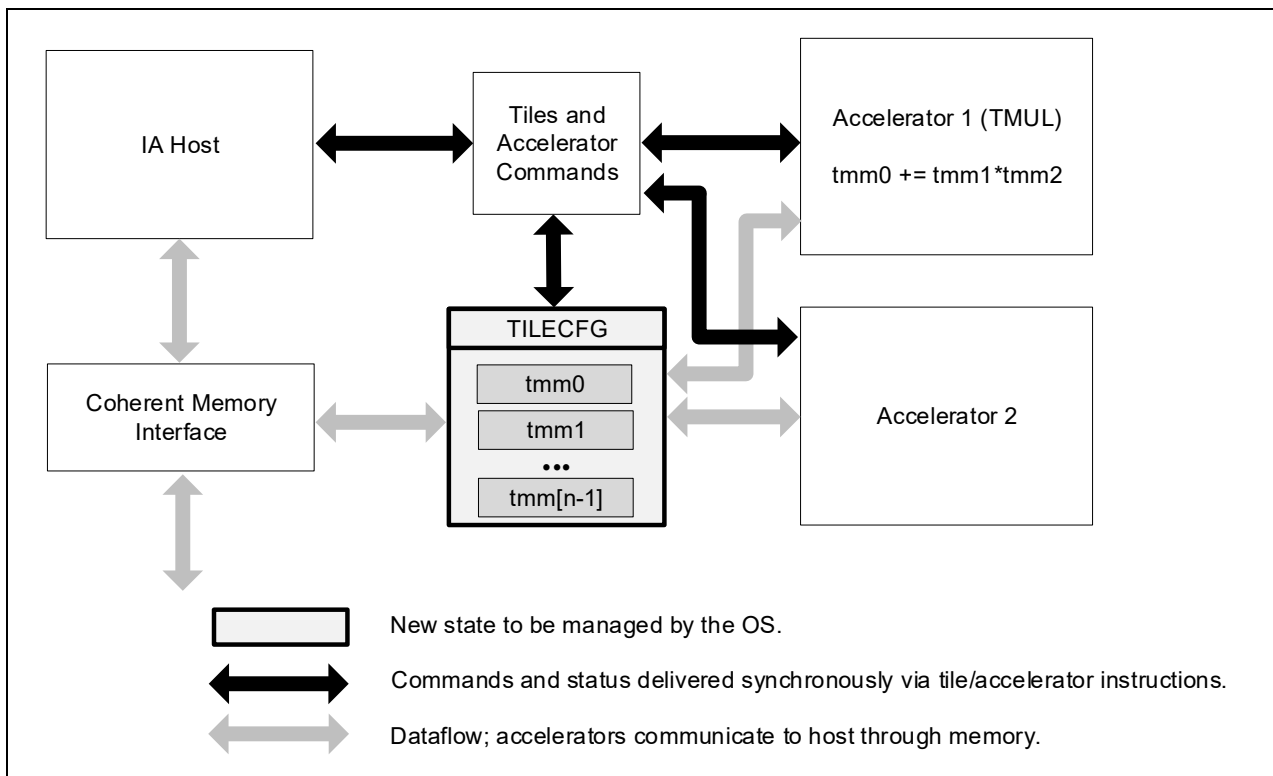


Figure 19-1. Intel® AMX Architecture

Figure 19-1 shows a conceptual diagram of the Intel AMX architecture. An Intel architecture host drives the algorithm, the memory blocking, loop indices and pointer arithmetic. Tile loads and stores and accelerator commands are sent to multi-cycle execution units. Status, if required, is reported back. Intel AMX instructions are synchronous in the Intel architecture instruction stream and the memory loaded and stored by the tile instructions is coherent with respect to the host's memory accesses. There are no restrictions on interleaving of Intel architecture and Intel AMX code or restrictions on the resources the host can use in parallel with Intel AMX (e.g., Intel AVX-512). There is also no architectural requirement on the Intel architecture compute capability of the Intel architecture host other than it supports 64-bit mode.

Intel AMX instructions use new registers and inherit basic behavior from Intel architecture in the same manner that Intel SSE and Intel AVX did. Tile instructions include loads and stores using the traditional Intel architecture register set as pointers. The TMUL instruction set (defined to be CPUID bits AMX_BF16 and AMX_INT8) only supports reg-reg operations.

TILECFG is programmed using the LDTILECFG instruction. The selected palette defines the available storage and general configuration while the rest of the memory data specifies the number of rows and column bytes for each tile. Consistency checks are performed to ensure the TILECFG matches the restrictions of the palette. A General Protection fault (#GP) is reported if the LDTILECFG fails consistency checks. A successful load of TILECFG with a palette_id other than 0 is represented in this document with TILES_CONFIGURED = 1. When the TILECFG is initialized (palette_id = 0), it is represented in the document as TILES_CONFIGURED = 0. Nearly all Intel AMX instructions will generate a #UD exception if TILES_CONFIGURED is not equal to 1; the exceptions are those that do TILECFG maintenance: LDTILECFG, STTILECFG, and TILERELEASE.

If a tile is configured to contain M rows by N column bytes, LDTILECFG will ensure that the metadata values are appropriate to the palette (e.g., that $M \leq 16$ and $N \leq 64$ for palette 1). The four M and N values can all be different as long as they adhere to the restrictions of the palette. Further dynamic checks are done in the tile and the TMUL instruction set to deal with cases where a legally configured tile may be inappropriate for the instruction operation. Tile registers can be set to 'invalid' by configuring the rows and colsb to '0'.

Tile loads and stores are strided accesses from the application memory to packed rows of data. Algorithms are expressed assuming row major data layout. Column major users should translate the terms according to their orientation.

TILELOAD* and TILESTORE* instructions are restartable and can handle (up to) 2*rows page faults per instruction. Restartability is provided by a **start_row** parameter in the TILECFG register.

The TMUL unit is conceptually a grid of fused multiply-add units able to read and write tiles. The dimensions of the TMUL unit (tmul_maxk and tmul_maxn) are enumerated similar to the maximum dimensions of the tiles (see "CPUID—CPU Identification" in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A for details).

The matrix multiplications in the TMUL instruction set compute $C[M][N] += A[M][K] * B[K][N]$. The M, N, and K values will cause the TMUL instruction set to generate a #UD exception if the dimensions do not match for matrix multiply or do not match the palette.

In Figure 19-2, the number of rows in tile B matches the K dimension in the matrix multiplication pseudocode. K dimensions smaller than that enumerated in the TMUL grid are also possible and any additional computation the TMUL unit can support will not affect the result.

The number of elements specified by colsb of the B matrix is also less than or equal to tmul_maxn. Any remaining values beyond that specified by the metadata will be set to zero.

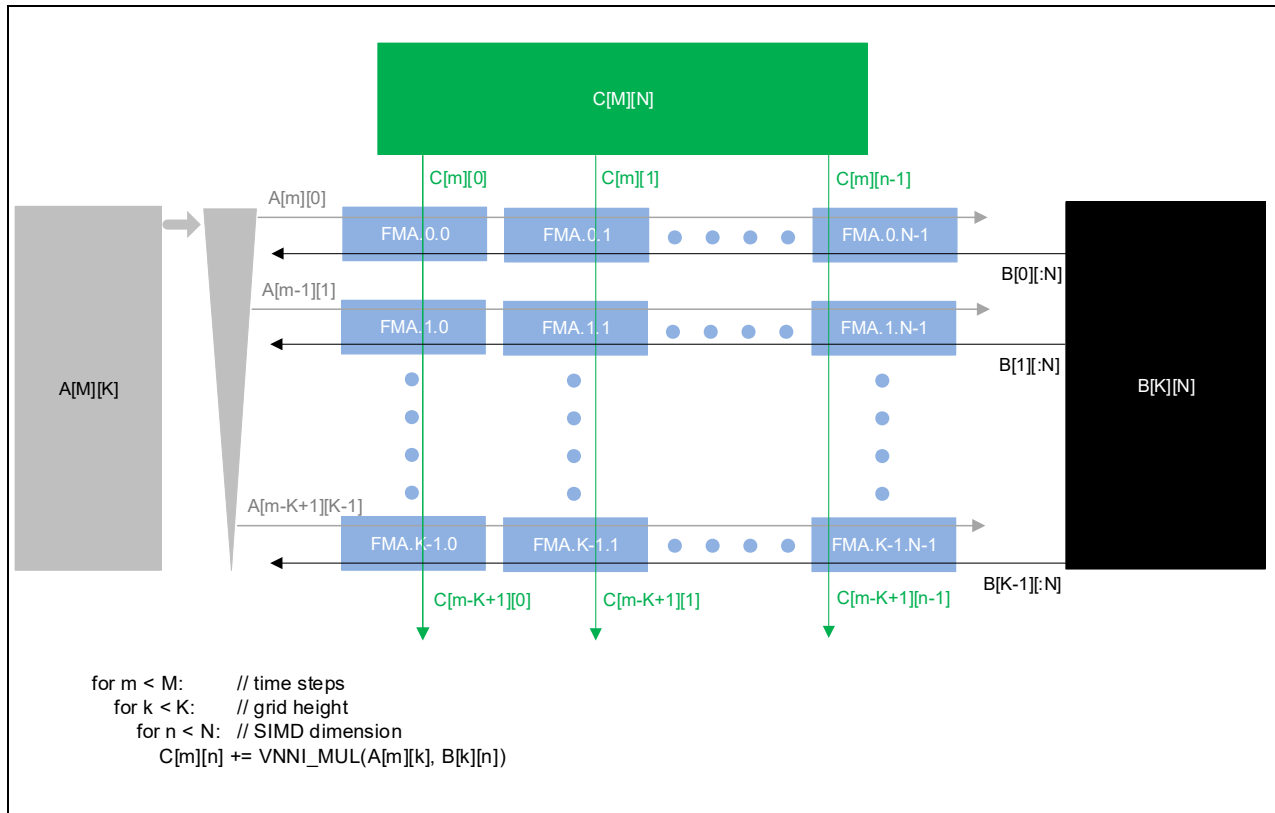


Figure 19-2. The TMUL Unit

The XSAVE feature set supports context management of the new state defined for Intel AMX. This support is described in Section 19.2.

19.1.1 Tile Architecture Details

The supported parameters for the tile architecture are reported via CPUID; this includes information about how the number of tile registers (`max_names`) can be configured (the palette). Configuring the tile architecture is intended to be done once when entering a region of tile code using the `LDTILECFG` instruction specifying the selected palette and describing in detail the configuration for each tile. Incorrect assignments will result in a General Protection fault (#GP). Successful `LDTILECFG` initializes (zeroes) `TILEDATA`.

Exiting a tile region is done with the `TILERELASE` instruction. It takes no parameters and invalidates all tiles (indicating that the data no longer needs any saving or restoring). Essentially, it is an optimization of `LDTILECFG` with an implicit palette of 0.

For applications that execute consecutive Intel AMX regions with differing configurations, `TILERELASE` is not required between them since the second `LDTILECFG` will clear all the data while loading the new configuration. There is no instruction set support for automatic nesting of tile regions, though with sufficient effort software can accomplish this by saving and restoring `TILEDATA` and `TILECFG` either through the XSAVE architecture or the Intel AMX instructions.

The tile architecture boots in its INIT state, with `TILECFG` and `TILEDATA` set to zero. A successfully executing `LDTILECFG` instruction to a non-zero palette sets the `TILES_CONFIGURED=1`, indicating the `TILECFG` is not in the INIT state. The `TILERELASE` instruction sets `TILES_CONFIGURED = 0` and initializes (zeroes) `TILEDATA`.

To facilitate handling of tile configuration data, there is a `STTILECFG` instruction. If the tile configuration is in the INIT state (`TILES_CONFIGURED == 0`), then `STTILECFG` will write 64 bytes of zeros. Otherwise `STTILECFG` will store the `TILECFG` to memory in the format used by `LDTILECFG`.

19.1.2 TMUL Architecture Details

The supported parameters for the TMUL architecture are reported via CPUID; see “CPUID—CPU Identification” in the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 2A, for details. These parameters include a maximum height (**tmul_maxk**) and a maximum SIMD dimension (**tmul_maxn**). The metadata that accompanies the srcdest, src1, and src2 tiles to the TMUL unit will be dynamically checked to see that they match the TMUL unit support for the data type and match the requirements of a meaningful matrix multiplication.

Figure 19-3 shows an example of the inner loop of an algorithm of using the TMUL architecture to compute a matrix multiplication. In this example, we use two result tiles, tmm0 and tmm1, from matrix C to accumulate the intermediate results. One tile from the A matrix (tmm2) is re-used twice as we multiply it by two tiles from the B matrix. The algorithm then advances pointers to load a new A tile and two new B tiles from the directions indicated by the arrows. An outer loop, not shown, adjusts the pointers for the C tiles.

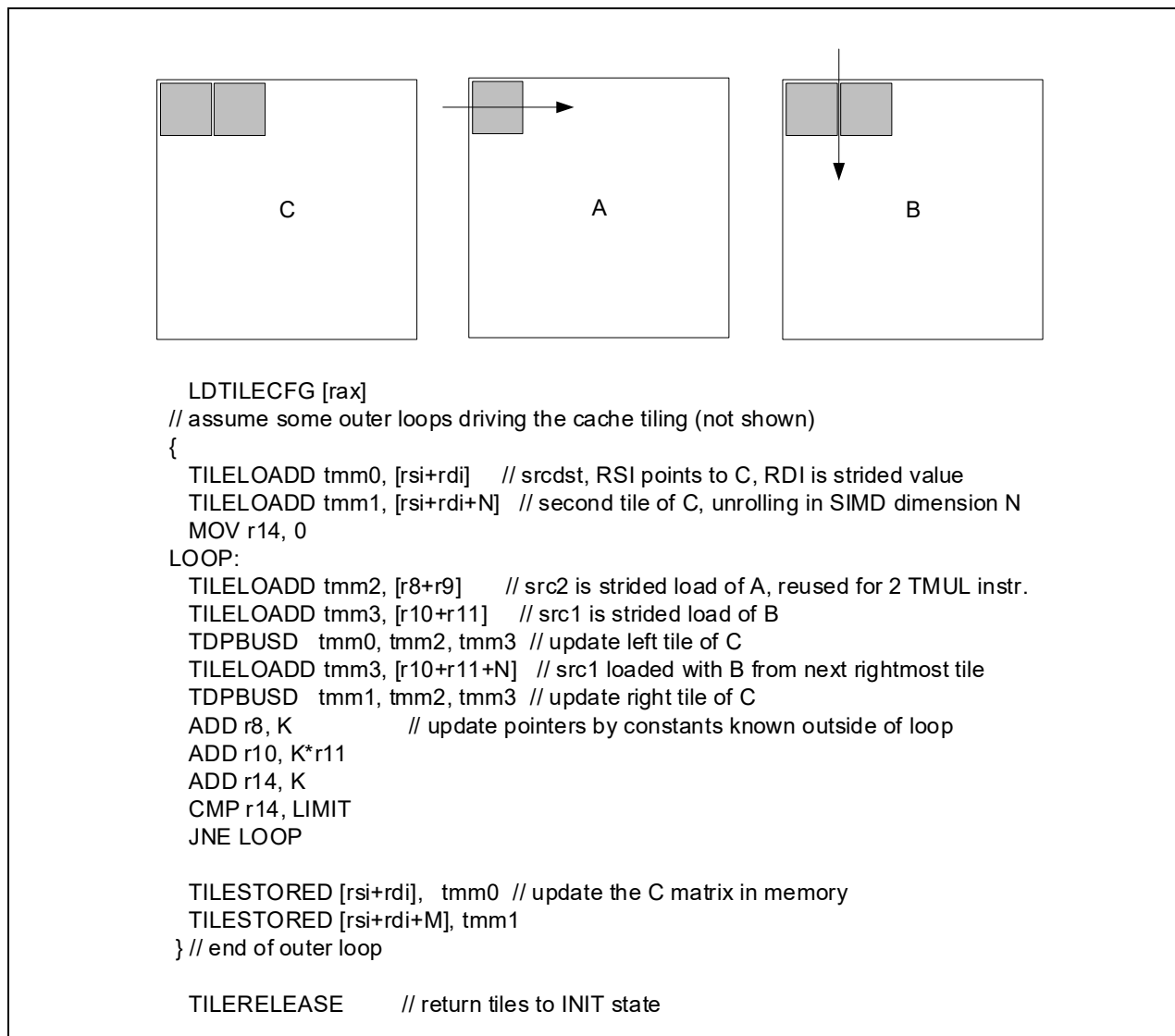


Figure 19-3. Matrix Multiply $C += A*B$

19.1.3 Handling of Tile Row and Column Limits

Intel AMX operations will zero any rows and any columns beyond the dimensions specified by TILECFG. Tile operations will zero the data beyond the configured number of column bytes as each row is written. For example, with 64-byte rows and a tile configured with 10 rows and 48 columns, an operation writing dword elements would write

each of the first 10 rows with 48 bytes of output/result data and zero the remaining 16 bytes in each row. Tile operations also fully zero any rows after the first 10 configured rows. When using a 1 KByte tile with 64-byte rows, there would be 16 rows, so in this example, the last 6 rows would also be zeroed.

Intel AMX instructions will always obey the metadata on reads and the zeroing rules on writes, and so a subsequent XSAVE would see zeros in the appropriate locations. Tiles that are not written by Intel AMX instructions between XRSTOR and XSAVE will write back with the same image they were loaded with regardless of the value of TILECFG.

19.1.4 Exceptions and Interrupts

Tile instructions are restartable so that operations that access strided memory can restart after page faults. To support restarting instructions after these events, the instructions store information in the **TILECFG.start_row** register. TILECFG.start_row indicates the row that should be used for restart; i.e., it indicates **next row after** the rows that have already been successfully loaded (on a TILELOAD) or written to memory (on a TILESTORE) and prevents repeating work that was successfully done.

The TMUL instruction set is not sensitive to the TILECFG.start_row value; this is due to there not being TMUL instructions with memory operands or any restartable faults.

19.2 RECOMMENDATIONS FOR SYSTEM SOFTWARE

Intel AMX is an XSAVE-enabled feature, meaning that it requires use of the XSAVE feature set for their enabling. Specifically, Intel AMX instructions and state are available only if system software has set CR4.OSXSAVE and also set XCR0[18:17] to 11B. In addition, use of Intel AMX instructions is disabled if system software has used extended feature disable (XFD) and set either IA32_XFD[17] or IA32_XFD[18] to 1. See Chapter 13, “Managing State Using the XSAVE Feature Set,” for more details.

NOTE

The first processors implementing Intel AMX will support setting IA32_XFD[18] but not IA32_XFD[17].

Once Intel AMX has been enabled, system software can disable it by clearing XCR0[18:17], by clearing CR4.OSXSAVE, or by setting either IA32_XFD[17] or IA32_XFD[18]. Before doing so, system software should first initialize AMX state (e.g., by executing TILERELEASE); maintaining AMX state in a non-initialized state may have negative power and performance implications and will prevent the execution of In-Field Scan tests. In addition, software should not rely on the state of the tile data after setting IA32_XFD[17] or IA32_XFD[18]; software should always reload or reinitialize the tile data after clearing IA32_XFD[17] and IA32_XFD[18].

System software should not use XFD to implement a “lazy restore” approach to management of the TILEDATA state component. This approach will **not** operate correctly for a variety of reasons. One is that the LDTILECFG and TILERELEASE instructions initialize TILEDATA and do not cause an #NM exception. Another is that an execution of XSAVE, XSAVEC, XSAVEOPT, or XSAVES by a user thread will save TILEDATA as initialized instead of the data expected by the user thread.

19.3 IMPLEMENTATION PARAMETERS

The parameters are reported via CPUID leaf 1DH. Index 0 reports all zeros for all fields.

```

define palette_table[id]:
    uint16_t total_tile_bytes
    uint16_t bytes_per_tile
    uint16_t bytes_per_row
    uint16_t max_names
    uint16_t max_rows

```

The tile parameters are set by LDTILECFG or XRSTOR* of TILECFG:

```

define tile[tid]:
    byte rows
    word colsb // bytes_per_row
    bool valid

```

19.4 HELPER FUNCTIONS

The helper functions used in Intel AMX instructions are defined below.

```

define write_row_and_zero(treg, r, data, nbytes):
    for j in 0 ...nbytes-1:
        treg.row[r].byte[j] := data.byte[j]

    // zero the rest of the row
    for j in nbytes ... palette_table[tilecfg.palette_id].bytes_per_row-1:
        treg.row[r].byte[j] := 0

define zero_upper_rows(treg, r):
    for i in r ... palette_table[tilecfg.palette_id].max_rows-1:
        for j in 0 ... palette_table[tilecfg.palette_id].bytes_per_row-1:
            treg.row[i].byte[j] := 0

define zero_tilecfg_start():
    tilecfg.start_row := 0

define zero_all_tile_data():
    if XCR0[TILEDATA]:
        b := CPUID.0DH.TILEDATA:EAX // size of feature
        for j in 0 ... b:
            TILEDATA.byte[j] := 0

```

```
define xcr0_supports_palette(palette_id):  
    if palette_id == 0:  
        return 1  
    elif palette_id == 1:  
        if XCR0[TILECFG] and XCR0[TILEDATA]:  
            return 1  
    return 0
```


In addition to transferring data to and from external memory, IA-32 processors can also transfer data to and from input/output ports (I/O ports). I/O ports are created in system hardware by circuitry that decodes the control, data, and address pins on the processor. These I/O ports are then configured to communicate with peripheral devices. An I/O port can be an input port, an output port, or a bidirectional port. Some I/O ports are used for transmitting data, such as to and from the transmit and receive registers, respectively, of a serial interface device. Other I/O ports are used to control peripheral devices, such as the control registers of a disk controller.

This chapter describes the processor's I/O architecture. The topics discussed include:

- I/O port addressing.
- I/O instructions.
- I/O protection mechanism.

20.1 I/O PORT ADDRESSING

The processor permits applications to access I/O ports in either of two ways:

- Through a separate I/O address space.
- Through memory-mapped I/O.

Accessing I/O ports through the I/O address space is handled through a set of I/O instructions and a special I/O protection mechanism. Accessing I/O ports through memory-mapped I/O is handled with the processor's general-purpose move and string instructions, with protection provided through segmentation or paging. I/O ports can be mapped so that they appear in the I/O address space or the physical-memory address space (memory mapped I/O) or both.

One benefit of using the I/O address space is that writes to I/O ports are guaranteed to be completed before the next instruction in the instruction stream is executed. Thus, I/O writes to control system hardware cause the hardware to be set to its new state before any other instructions are executed. See Section 20.6, "Ordering I/O," for more information on serializing of I/O operations.

20.2 I/O PORT HARDWARE

From a hardware point of view, I/O addressing is handled through the processor's address lines. For the P6 family, Pentium 4, and Intel Xeon processors, the request command lines signal whether the address lines are being driven with a memory address or an I/O address; for Pentium processors and earlier IA-32 processors, the M/IO# pin indicates a memory address (1) or an I/O address (0). When the separate I/O address space is selected, it is the responsibility of the hardware to decode the memory-I/O bus transaction to select I/O ports rather than memory. Data is transmitted between the processor and an I/O device through the data lines.

20.3 I/O ADDRESS SPACE

The processor's I/O address space is separate and distinct from the physical-memory address space. The I/O address space consists of 2^{16} (64K) individually addressable 8-bit I/O ports, numbered 0 through FFFFH. I/O port addresses 0F8H through 0FFH are reserved. Do not assign I/O ports to these addresses. The result of an attempt to address beyond the I/O address space limit of FFFFH is implementation-specific; see the Developer's Manuals for specific processors for more details.

Any two consecutive 8-bit ports can be treated as a 16-bit port, and any four consecutive ports can be a 32-bit port. In this manner, the processor can transfer 8, 16, or 32 bits to or from a device in the I/O address space. Like words in memory, 16-bit ports should be aligned to even addresses (0, 2, 4, ...) so that all 16 bits can be transferred in a

single bus cycle. Likewise, 32-bit ports should be aligned to addresses that are multiples of four (0, 4, 8, ...). The processor supports data transfers to unaligned ports, but there is a performance penalty because one or more extra bus cycle must be used.

The exact order of bus cycles used to access unaligned ports is undefined and is not guaranteed to remain the same in future IA-32 processors. If hardware or software requires that I/O ports be written to in a particular order, that order must be specified explicitly. For example, to load a word-length I/O port at address 2H and then another word port at 4H, two word-length writes must be used, rather than a single doubleword write at 2H.

Note that the processor does not mask parity errors for bus cycles to the I/O address space. Accessing I/O ports through the I/O address space is thus a possible source of parity errors.

20.3.1 Memory-Mapped I/O

I/O devices that respond like memory components can be accessed through the processor’s physical-memory address space (see Figure 20-1). When using memory-mapped I/O, any of the processor’s instructions that reference memory can be used to access an I/O port located at a physical-memory address. For example, the MOV instruction can transfer data between any register and a memory-mapped I/O port. The AND, OR, and TEST instructions may be used to manipulate bits in the control and status registers of a memory-mapped peripheral device.

Certain instructions may take an exception or VM exit after completing a memory access (either a read or a write) to a memory-mapped I/O address. This exception or VM exit could be due to the instruction performing multiple memory accesses (e.g., MOVS, PUSH mem, POP mem, PUSHAD, etc.) or could be due to the ordering of exceptions or VM exits within the instruction (e.g., a DIV mem that takes a #DE or a CALL that causes a task switch VM exit). If software later re-executes that instruction (e.g., after an IRET or VMRESUME), the MMIO (memory-mapped I/O) access may occur again. If the memory-mapped I/O access has a side-effect, that side-effect may be executed each time the memory-mapped I/O access occurs. If that is problematic, software must ensure that exceptions or VM exits do not occur after accessing the MMIO.

When using memory-mapped I/O, caching of the address space mapped for I/O operations must be prevented. With the Pentium 4, Intel Xeon, and P6 family processors, caching of I/O accesses can be prevented by using memory type range registers (MTRRs) to map the address space used for the memory-mapped I/O as uncacheable (UC). See Chapter 14, “Memory Cache Control,” in the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 3A, for a complete discussion of the MTRRs.

The Pentium and Intel486 processors do not support MTRRs. Instead, they provide the KEN# pin, which when held inactive (high) prevents caching of all addresses sent out on the system bus. To use this pin, external address decoding logic is required to block caching in specific address spaces.

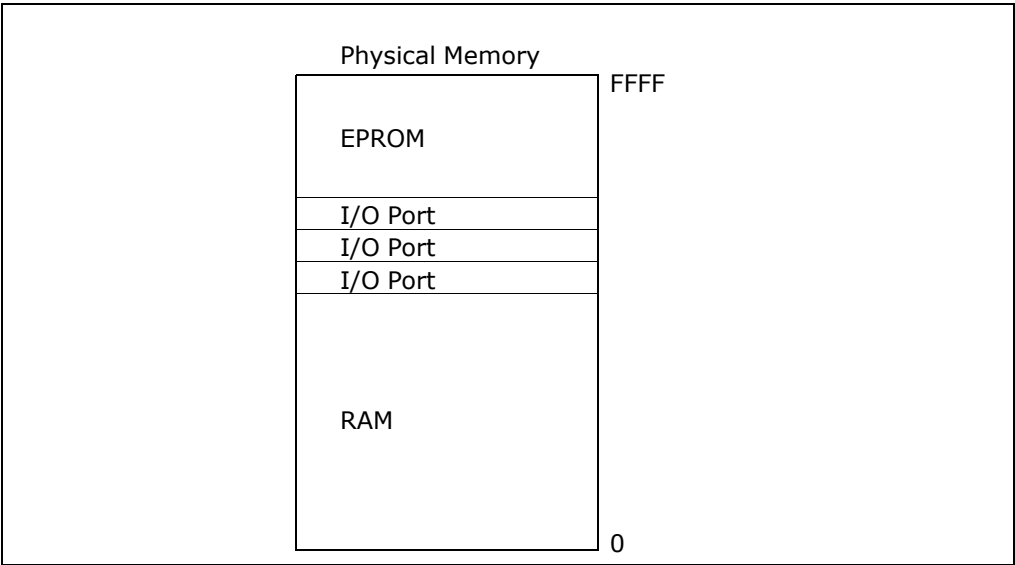


Figure 20-1. Memory-Mapped I/O

All the IA-32 processors that have on-chip caches also provide the PCD (page-level cache disable) flag in page table and page directory entries. This flag allows caching to be disabled on a page-by-page basis. See “Page-Directory and Page-Table Entries” in Chapter 4 of the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 3A.

20.4 I/O INSTRUCTIONS

The processor’s I/O instructions provide access to I/O ports through the I/O address space. (These instructions cannot be used to access memory-mapped I/O ports.) There are two groups of I/O instructions:

- Those that transfer a single item (byte, word, or doubleword) between an I/O port and a general-purpose register.
- Those that transfer strings of items (strings of bytes, words, or doublewords) between an I/O port and memory.

The register I/O instructions IN (input from I/O port) and OUT (output to I/O port) move data between I/O ports and the EAX register (32-bit I/O), the AX register (16-bit I/O), or the AL (8-bit I/O) register. The address of the I/O port can be given with an immediate value or a value in the DX register.

The string I/O instructions INS (input string from I/O port) and OUTS (output string to I/O port) move data between an I/O port and a memory location. The address of the I/O port being accessed is given in the DX register; the source or destination memory address is given in the DS:ESI or ES:EDI register, respectively.

When used with the repeat prefix REP, the INS and OUTS instructions perform string (or block) input or output operations. The repeat prefix REP modifies the INS and OUTS instructions to transfer blocks of data between an I/O port and memory. Here, the ESI or EDI register is incremented or decremented (according to the setting of the DF flag in the EFLAGS register) after each byte, word, or doubleword is transferred between the selected I/O port and memory.

See the references for IN, INS, OUT, and OUTS in Chapter 3 and Chapter 4 of the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volumes 2A, 2B, 2C, & 2D, for more information on these instructions.

20.5 PROTECTED-MODE I/O

When the processor is running in protected mode, the following protection mechanisms regulate access to I/O ports:

- When accessing I/O ports through the I/O address space, two protection devices control access:
 - The I/O privilege level (IOPL) field in the EFLAGS register.
 - The I/O permission bit map of a task state segment (TSS).
- When accessing memory-mapped I/O ports, the normal segmentation and paging protection and the MTRRs (in processors that support them) also affect access to I/O ports. See Chapter 6, “Protection,” and Chapter 14, “Memory Cache Control,” in the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 3A, for a complete discussion of memory protection.

The following sections describe the protection mechanisms available when accessing I/O ports in the I/O address space with the I/O instructions.

20.5.1 I/O Privilege Level

In systems where I/O protection is used, the IOPL field in the EFLAGS register controls access to the I/O address space by restricting use of selected instructions. This protection mechanism permits the operating system or executive to set the privilege level needed to perform I/O. In a typical protection ring model, access to the I/O address space is restricted to privilege levels 0 and 1. Here, the kernel and the device drivers are allowed to perform I/O, while less privileged device drivers and application programs are denied access to the I/O address space. Application programs must then make calls to the operating system to perform I/O.